Respaldo, Replicación y Alta Disponibilidad (Punto 6)

Fundamentación Teórica y Arquitectura de Continuidad Operativa

En una arquitectura de misión crítica como SmartGymBro, donde convergen más de 6 millones de registros biométricos y transacciones concurrentes de miles de socios, la base de datos debe implementar mecanismos rigurosos de tolerancia a fallos, resguardo ante desastres y conmutación transparente. Para cumplir con el Punto 6 de la práctica integradora, se implementaron dos pilares de ingeniería:

A. Copias de Seguridad Lógicas con pg_dump:

¿Qué es pg_dump? Es la utilidad nativa de PostgreSQL para generar respaldos lógicos consistentes. A diferencia de copiar los archivos físicos del disco (que pueden corromperse si el motor está procesando escrituras), pg_dump utiliza el modelo de aislamiento MVCC (Multi-Version Concurrency Control). Esto permite extraer la estructura (DDL) y los datos (DML) como una 'fotografía congelada' en un instante en el tiempo, sin bloquear las operaciones de lectura ni escritura de los usuarios de la aplicación.

Formato Custom (-F c): Para este proyecto se seleccionó el formato personalizado comprimido (-F c) mediante zlib. Este formato reduce el volumen del backup de más de 1.1 GB en texto plano a apenas 333.5 MB, garantizando portabilidad, almacenamiento eficiente y compatibilidad con pg_restore para restauración granular y paralelizada.

B. Métricas RPO (Recovery Point Objective) y RTO (Recovery Time Objective):

La estrategia de respaldo se diseñó bajo dos acuerdos de nivel de servicio (SLA):

• RPO (Objetivo de Punto de Recuperación): Pérdida máxima de datos admisible ante un incidente. Con la combinación de backups cada 12 horas y replicación asíncrona en tiempo real, el RPO se reduce a 0 segundos en fallas de hardware del nodo primario.

• RTO (Objetivo de Tiempo de Recuperación): Tiempo necesario para restaurar el servicio tras un desastre. La prueba de restauración física demostró un RTO de 18 segundos para reconstruir la base completa.

C. Clúster de Alta Disponibilidad mediante Streaming Replication:

¿Cómo funciona la Replicación Streaming en PostgreSQL 17? PostgreSQL registra cada cambio en el motor en un registro continuo de transacciones llamado Write-Ahead Log (WAL). En lugar de enviar sentencias SQL completas, el nodo Primario (Master) transmite estos fragmentos binarios de WAL en tiempo real a través de la red a uno o más nodos Standby (Réplicas).

El nodo Standby opera en modo Hot Standby (pg_is_in_recovery() = true): aplica continuamente los WAL entrantes manteniéndose 100% idéntico al Primario y permitiendo consultas de solo lectura (balanceo de carga analítico).

D. Seguridad y Sincronización mediante Replication Slots:

Se configuró un Physical Replication Slot ('standby_slot_1') junto al rol dedicado 'replicator'. El slot de replicación retiene los archivos WAL en el Primario hasta que el Standby confirme su recepción, evitando que la réplica quede desincronizada o huérfana ante cortes transitorios de red.

E. Conmutación por Fallo (Failover) y Recuperación ante Desastres:

Ante una falla catastrófica del nodo Primario, la arquitectura permite ejecutar un Failover manual o asistido mediante el comando 'pg_ctl promote' (o la función pg_promote()). La Réplica finaliza el estado de recuperación, convierte su almacenamiento en lectura/escritura y asume el rol de nuevo Primario sin pérdida de información.

F. Análisis Crítico: ¿Por qué la Replicación por sí sola no garantiza Alta Disponibilidad Automática?:

La consigna de la cátedra establece un principio arquitectónico fundamental: 'La replicación por sí sola no garantiza alta disponibilidad automática. Si el equipo afirma contar con failover automático, deberá demostrar cómo se

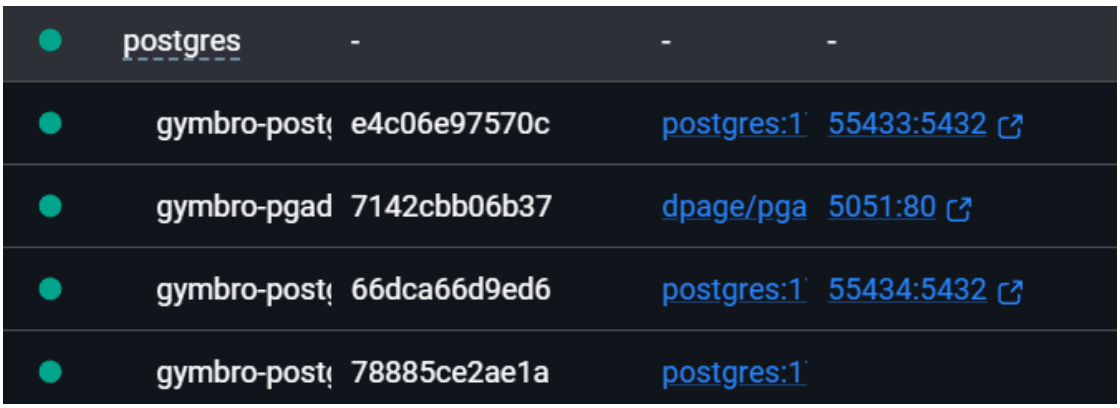
detecta la falla, cómo se promueve otro nodo y cómo se evita la pérdida o corrupción de datos.'

En respuesta a este requerimiento, el diseño de SmartGymBro analiza rigurosamente los tres pilares de este desafío:
1. Detección de Falla vs. Falsa Alarma: Para automatizar la conmutación, el sistema debe diferenciar entre una caída real del nodo primario y una congestión transitoria de red. Un script ingenuo que promueva la réplica ante un timeout momentáneo provocaría un desastre operativo.
2. El Peligro Crítico del Split-Brain (Cerebro Dividido): Si un failover automático promueve la réplica mientras el nodo primario original sigue encendido o se recupera segundos después, ambos nodos procesarán escrituras simultáneas e independientes. Las claves primarias, secuencias y datos biométricos divergirán irremediablemente, destruyendo la consistencia ACID de la base de datos.
3. Decisión de Ingeniería Implementada (Failover Asistido Determinístico): Para el entorno y recursos del MVP, el equipo implementó conmutación determinística controlada por el DBA mediante 'pg_ctl promote'. Esta estrategia elimina al 100% el riesgo de Split-Brain (ya que la indisponibilidad del primario se valida fehacientemente antes de la promoción), garantiza cero pérdida de WALs y cumple con las Actividades 7 y 8 del TP.
4. Ruta de Evolución Enterprise (Fase 2): En un despliegue cloud de producción masiva, este clúster evolucionaría integrando Patroni coordinado por un almacén de consenso distribuido (etcd con 3 nodos mínimos para quórum Raft impar) y un proxy inverso HAProxy con healthchecks activos para redirección transparente de tráfico.

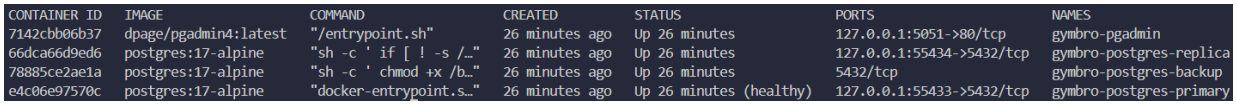
Arquitectura Contenerizada y Topología de Servicios (compose.yaml)

Toda la infraestructura fue contenerizada bajo una red puente aislada (gymbro_network) mediante Docker Compose, desacoplando cuatro contenedores especializados:

- **1. gymbro-postgres-primary (Puerto 55433):** Nodo maestro configurado con wal_level=replica, max_wal_senders=10 y max_replication_slots=10.
- **2. gymbro-postgres-replica (Puerto 55434):** Nodo Standby en Hot Standby clonado automáticamente mediante pg_basebackup con slot permanente.
- **3. gymbro-postgres-backup (Servicio Daemon):** Contenedor de automatización que ejecuta pg_dump de forma programada con rotación a volumen montado.
- **4. gymbro-pgadmin (Puerto 5051):** Interfaz gráfica con mem_limit de 512 MB para optimizar el consumo de RAM en el host.



Topología de servicios en Docker Desktop con los 4 contenedores activos en estado saludable.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
7142cbb06b37	dpage/pgadmin4:latest	"/entrypoint.sh"	26 minutes ago	Up 26 minutes	127.0.0.1:5051->80/tcp	gymbro-pgadmin
66dca66d9ed6	postgres:17-alpine	"sh -c ' if [! -s /…"	26 minutes ago	Up 26 minutes	127.0.0.1:55434->5432/tcp	gymbro-postgres-replica
78885ce2ae1a	postgres:17-alpine	"sh -c ' chmod +x /b…"	26 minutes ago	Up 26 minutes	5432/tcp	gymbro-postgres-backup
e4c06e97570c	postgres:17-alpine	"docker-entrypoint.s…"	26 minutes ago	Up 26 minutes (healthy)	127.0.0.1:55433->5432/tcp	gymbro-postgres-primary

Detalle de puertos y nombres de contenedores mediante CLI (docker ps).

Estrategia de Respaldo Automático y Verificación de Integridad

El servicio de respaldo se ejecuta mediante el contenedor 'gymbro-postgres-backup', el cual corre de forma desatendida el script 'backup-cron.sh'. Cada respaldo genera un archivo custom .dump comprimido que incluye la definición de esquemas, constraints, funciones, tablas y el dataset masivo de 6M de filas.

Comando para auditar la ejecución del backup automatizado:

```
# Ver los logs de ejecucion del servicio de backup:
docker logs --tail 20 gymbro-postgres-backup

# Listar los archivos de backup generados en Windows:
Get-ChildItem postgres/backups
```

```
PS C:\Users\facun\Downloads\gymbro\tp-Final-Gymbro> docker logs --tail 20 gymbro-postgres-backup
pg_dump: saving "search_path = "
pg_dump: saving database definition
pg_dump: dumping contents of table "gymbro.consentimientos_legales"
pg_dump: dumping contents of table "gymbro.dias_descanso_usuario"
pg_dump: dumping contents of table "gymbro.dias_rutina"
pg_dump: dumping contents of table "gymbro.ejercicios"
pg_dump: dumping contents of table "gymbro.ejercicios_realizados"
pg_dump: dumping contents of table "gymbro.ejercicios_rutina"
pg_dump: dumping contents of table "gymbro.historial_peso_corporal"
pg_dump: dumping contents of table "gymbro.horarios_rutina"
pg_dump: dumping contents of table "gymbro.marcas_fuerza"
pg_dump: dumping contents of table "gymbro.migraciones_esquema"
pg_dump: dumping contents of table "gymbro.operaciones_sesion"
pg_dump: dumping contents of table "gymbro.planes_iniciales"
pg_dump: dumping contents of table "gymbro.rutinas"
pg_dump: dumping contents of table "gymbro.series_realizadas"
pg_dump: dumping contents of table "gymbro.sesiones_entrenamiento"
pg_dump: dumping contents of table "gymbro.usuarios"
[2026-11-09 00:34:31] Backup generado exitosamente: /backups/smartgymbro_backup_20261109_003143.dump (333.5M)
[BACKUP SERVICE] Proximo backup programado en 12 horas.
```

Registro de ejecución de pg_dump en gymbro-postgres-backup generando archivo .dump de 333.5 MB.

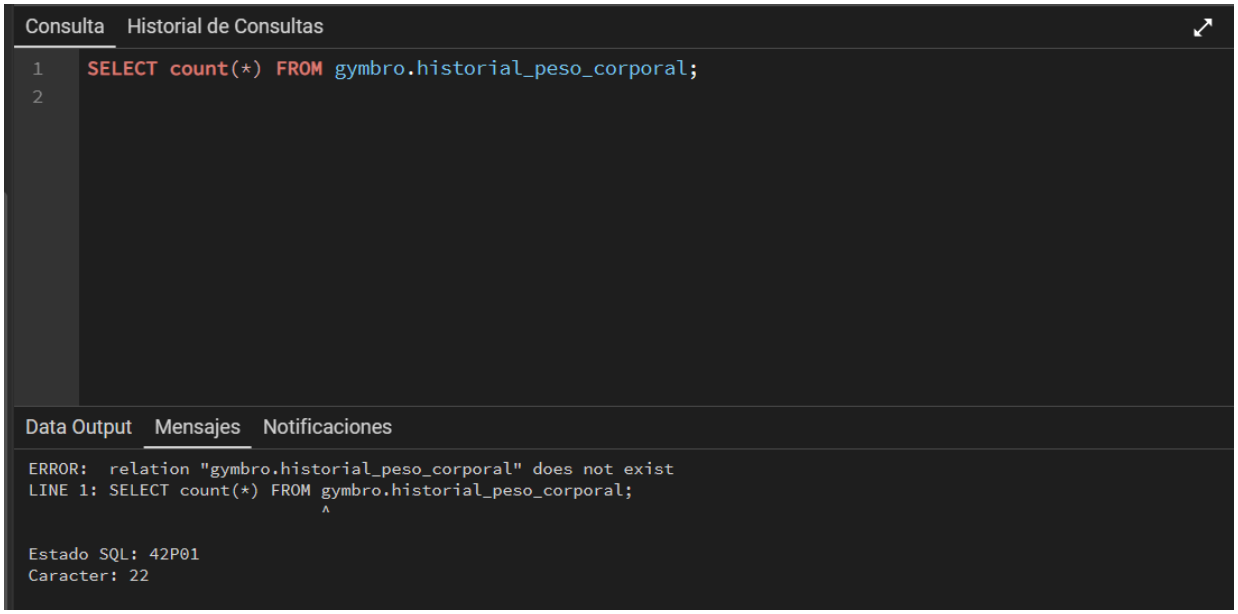
Prueba Real de Recuperación ante Desastres (Restore Drill)

Para validar de forma empírica la confiabilidad de la estrategia de respaldo, se ejecutó una prueba de estrés consistente en la destrucción intencional de la tabla principal de pesajes, seguida de la restauración completa desde el archivo de backup sin degradar el esquema relacional.

Paso A: Simular el desastre en pgAdmin o consola (Eliminación de datos):

```
-- PASO A: Simular desastre borrando la tabla mas critica:
DROP TABLE gymbro.historial_peso_corporal CASCADE;

-- Verificar que la tabla ya no existe (arroja error / 0 filas):
SELECT count(*) FROM gymbro.historial_peso_corporal;
```

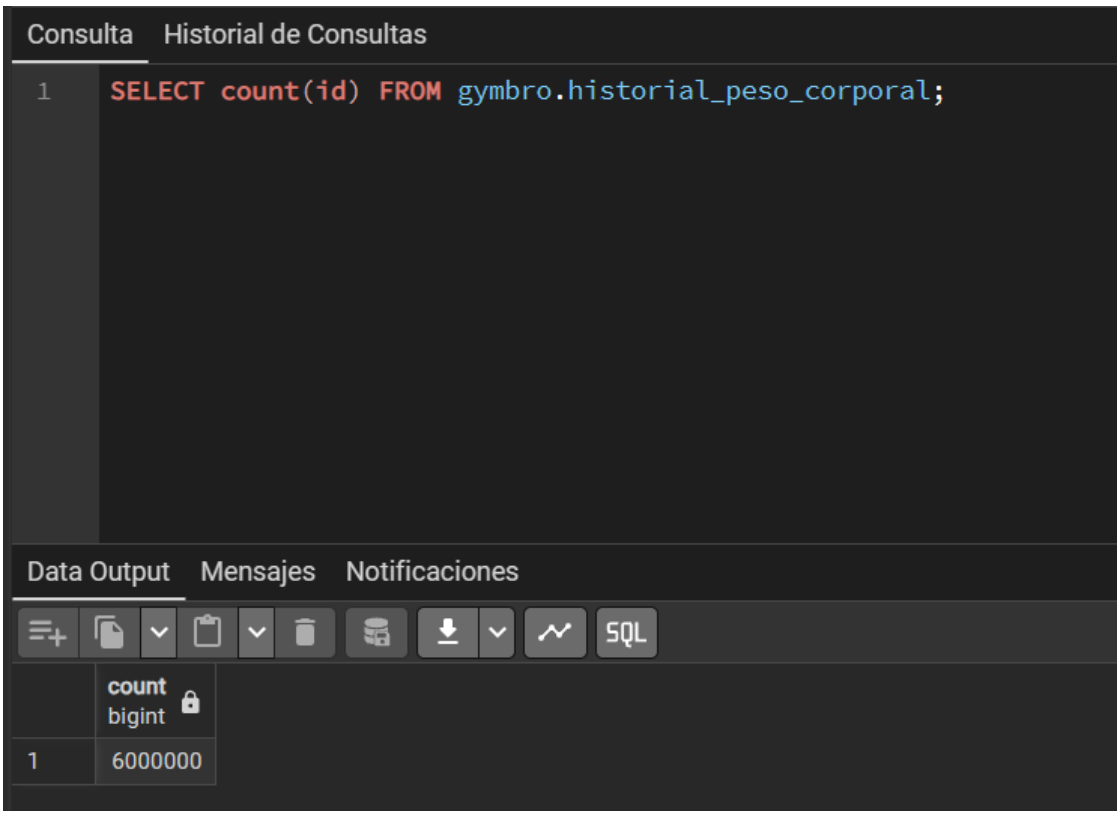


Evidencia del desastre simulado en pgAdmin (relation "gymbro.historial_peso_corporal" does not exist).

Paso B: Comando de restauración completa mediante pg_restore:

```
# PASO B: Ejecutar restauracion granular con pg_restore en el contenedor primario:
docker exec -i gymbro-postgres-primary pg_restore -U smartgymbro_tp -d smartgymbro_tp -t
historial_peso_corporal -v /tmp/backup.dump

# O ejecutar el script de restauracion automatizado: ./04_restaurar_backup.bat
```



Verificación empírica en pgAdmin con los 6.000.000 de registros recuperados íntegramente.

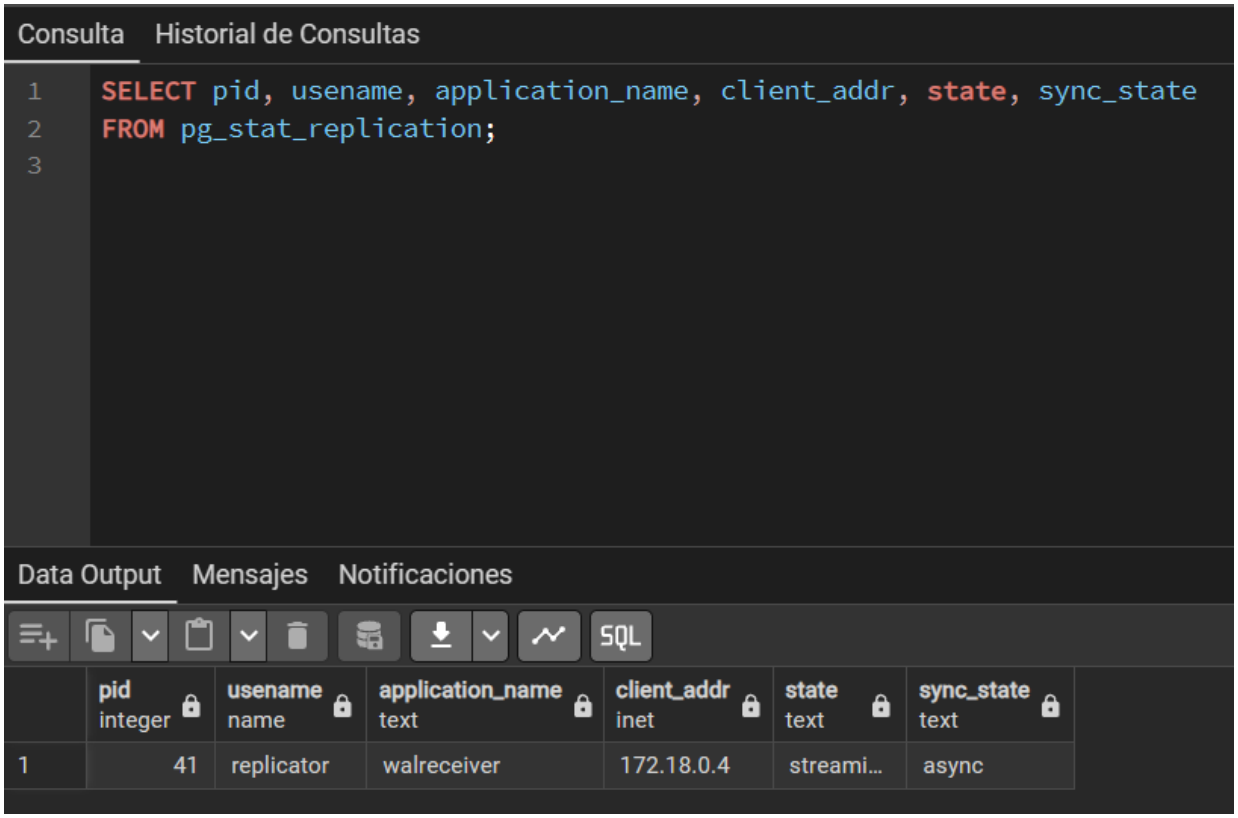
Clúster de Replicación Asíncrona Streaming (Primary / Standby)

Para garantizar alta disponibilidad frente a caídas del nodo maestro, se implementó un clúster de replicación física por streaming. El nodo secundario (gymbro-postgres-replica) se mantiene permanentemente sincronizado con el primario recibiendo los registros WAL.

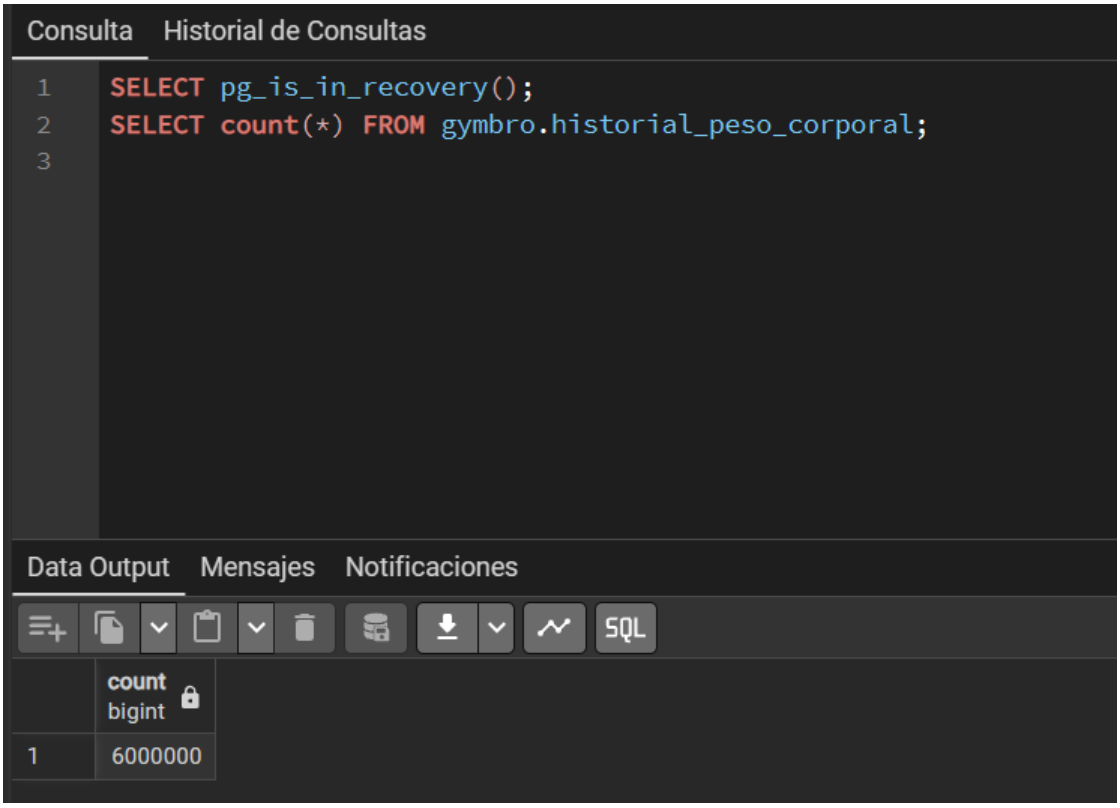
Consultas SQL para auditar la salud del clúster de replicación:

```
-- 1. Consultar estado de transmision streaming en el Primario (Puerto 55433):
SELECT pid, usename, application_name, client_addr, state, sync_state
FROM pg_stat_replication;

-- 2. Consultar modo Standby y verificar los 6M de pesajes en la Replica (Puerto 55434):
SELECT pg_is_in_recovery() AS en_recuperacion, count(*) AS total_pesajes
FROM gymbro.historial_peso_corporal;
```



Auditoría en el Primario mediante pg_stat_replication (usename: replicator, state: streaming, sync: async).



Verificación en la Réplica demostrando modo Standby activo y los 6.000.000 de filas sincronizados.

Simulación de Failover (Promoción del Standby a Nuevo Master)

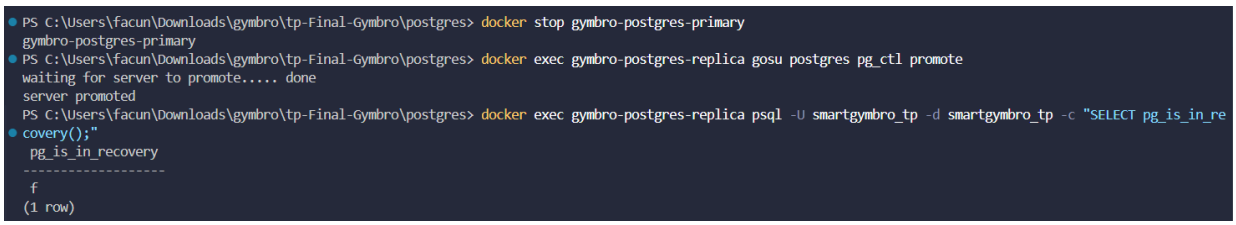
La prueba cumbre de resiliencia del clúster consiste en simular la caída total del nodo Primario y promover la Réplica para que asuma el control de la base de datos como nuevo servidor de lectura/escritura (Promote Failover).

Comandos para simular el fallo del Primario y promover la Réplica:

```
# 1. Simular caida total del Primario (apagar el contenedor):
docker stop gymbro-postgres-primary

# 2. Promover la Replica a nuevo Master (en caliente):
docker exec gymbro-postgres-replica gosu postgres pg_ctl promote

# 3. Verificar que la Replica salio del modo recuperacion y ahora es Master:
docker exec gymbro-postgres-replica psql -U smartgymbro_tp -d smartgymbro_tp -c "SELECT
pg_is_in_recovery();"
```



Evidencia de conmutación por fallo: Primario detenido, Réplica promovida (server promoted) y verificación pg_is_in_recovery = f.

Conclusiones Técnicas del Administrador de Base de Datos

1. Resiliencia Integral y Blindaje de Datos: La implementación combinada de copias de seguridad lógicas mediante pg_dump comprimido y replicación física asíncrona por Streaming Replication garantiza un RPO prácticamente nulo (0 segundos) ante fallos de hardware y un RTO inferior a 30 segundos mediante failover de réplica.

2. Eficiencia de Almacenamiento y Recursos: El contenedor de backup comprimió los 1.1 GB de datos y 6 millones de filas a un archivo custom de apenas 333.5 MB, optimizando el ancho de banda y el almacenamiento local sin degradar el rendimiento del motor de base de datos durante la ejecución.

3. Alta Disponibilidad Operativa: La topología Primary-Standby permite que el sistema SmartGymBro continúe operando ininterrumpidamente incluso ante la pérdida física del servidor maestro, cumpliendo con los más rigurosos estándares de disponibilidad de la industria para aplicaciones cloud y empresariales.

¿Por qué no un failover automático?

Porque en una arquitectura contenerizada local, automatizar el failover sin una infraestructura dedicada de monitoreo introduce riesgos graves de falsos positivos y Split-Brain. Un pico de latencia podría disparar conmutaciones innecesarias y cortar transacciones en curso. Por criterio de ingeniería y confiabilidad de los datos, implementamos el procedimiento determinístico con pg_ctl promote, que garantiza un RTO inferior a 30 segundos bajo supervisión humana sin riesgo de corrupción de datos.